PSYC 51.17: Models of Language and Communication - Week 3

Word Embeddings: Word2Vec, GloVe, FastText

Lecture 11: The Neural Revolution in NLP

PSYC 51.17: Models of Language and Communication - Week 3

Winter 2026

Winter 2026

Today's Lecture

1. **The 2013 Revolution: Word2Vec**
2. **Word2Vec Architectures: CBOW & Skip-gram**
3. **GloVe: Global Vectors**
4. **FastText: Subword Information**
5. **Comparison & Evaluation**
6. **Applications & Best Practices**

Goal: Understand how neural methods learn dense word representations

Winter 2026

From Count-Based to Prediction-Based

Count-Based (LSA, LDA):

- Build co-occurrence matrix
- Apply matrix factorization
- Global statistics
- Linear relationships
- Interpretable factors

Prediction-Based (Word2Vec):

- Predict context from word
- Train neural network
- Local context windows
- Non-linear relationships
- Learned representations

Word2Vec: The Revolution

2013: Everything changed

Key Innovation:

- Shallow neural network
- Predict context from word (CBOW)
- Predict word from context (Skip-gram)

Famous Results:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

Other Examples:

- $\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \vec{\text{Rom}}$
- $\vec{\text{walking}} - \vec{\text{walk}} + \vec{\text{swim}} \approx \text{swi}$

Word2Vec: Core Intuition

Words that appear in similar contexts should have similar representations

Example Context Window

"The quick brown **[TARGET]** jumped over the lazy dog"

Context: [The, quick, brown, jumped, over, the, lazy, dog]

Target: fox

Key Idea:

- Train a model to predict target from context (or vice versa)
- The learned **weights** become our word vectors!

Context Windows: Worked Example

Sentence: "The cat sat on the mat"

Window size = 2 (2 words on each side)

```
Position 0: "The" → Context: [cat, sat]
```

Skip-gram training pairs (target → context):

```
(The, cat), (The, sat)
```

Result: Words appearing in similar contexts get similar vectors!

CBOW: Continuous Bag of Words

Predict the center word from context words

```
Context words:
 "the"
 "cat" →
Average →
"sat"
 "on" →
```

Architecture:

1. Input: One-hot vectors of context words
2. Hidden: Average of input embeddings
3. Output: Softmax over

Skip-gram: The Inverse Approach

Predict context words from the center word

```
Center word: Predict context:
 → "the"
 → "cat"
 "sat" → Hidden →
```

Architecture:

1. Input: One-hot vector of center word
2. Hidden: Word embedding
3. Output: Softmax predicting each context word

$$\sum_{w=1} \exp(v_w v_{w_I})$$

For 100k vocabulary: Need to compute 100k exponentials per training example!

Solution: Negative Sampling

Instead of predicting across all words, create a binary classification:

- **Positive sample:** Actual context word (label = 1)
- **Negative samples:** K random words (label = 0)

$$\log \sigma(v_{w_O}^T v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-v_{w_i}^T v_{w_I})] \quad (9)$$

Typical K: 5-20 for large datasets, 2-5 for small

Negative Sampling: Concrete Example

Training pair: ("cat", "sat") - cat is center, sat is context

```
# Positive sample: Does "sat" appear near "cat"? YES (label=1)
positive_pair = ("cat", "sat", label=1)
```

Sampling distribution: $P_n(w) \propto \text{freq}(w)^{0.75}$

The 0.75 power gives rare words slightly higher probability of being sampled as negatives.

Word2Vec in Practice

```python
from gensim.models import Word2Vec
import gensim.downloader as api

# Load pre-trained model (300 dimensions, 3B words)
model = api.load('word2vec-google-news-300')

# Find similar words
similar = model.most_similar('computer', topn=5)
print(similar)
# Output: [('computers', 0.72), ('laptop', 0.69), ('PC', 0.68), ...]

# Word analogies: king - man + woman = ?
result = model.most_similar(
 positive=['king', 'woman'],
 negative=['man'],
 topn=1
```


continued...

Word2Vec in Practice

```
)
print(result)
# Output: [('queen', 0.71)]

# Train your own model
sentences = [
 ['the', 'cat', 'sat', 'on', 'the', 'mat'],
 ['the', 'dog', 'ran', 'in', 'the', 'park'],
 #  ...  more sentences
]

custom_model = Word2Vec(
 sentences,
 vector_size=100, # embedding dimension
 window=5, # context window
 min_count=1, # ignore rare words
```


...continued...

Word2Vec in Practice

```
 sg=1, # 1=skip-gram, 0=CBOW
 negative=5, # negative sampling
 workers=4 # parallel threads
)
```

Winter 2026

...continued

GloVe: Global Vectors for Word Representation

Combining the best of count-based and prediction-based methods

Motivation:

- Word2Vec uses **local** context windows
- LSA uses **global** co-occurrence statistics

Co-occurrence Example:

Probe word	P(word|ice)	P(word|steam)	Ratio

GloVe: The Ratio Intuition

Why ratios matter more than raw probabilities:

```
Given words: "ice" and "steam"
Probe with: "solid", "gas", "water"

```

The insight: These ratios distinguish relevant context words from irrelevant ones!

GloVe learns vectors where: $\frac{\vec{w}_{\text{ice}}^T \vec{w}_{\text{solid}}}{\vec{w}_{\text{steam}}^T \vec{w}_{\text{solid}}} \approx 8.9$

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(\vec{w}_i^T \tilde{\vec{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where:

- X_{ij} = number of times word j appears in context of word i
- $\vec{w}_i$ = word vector for word i
- $\tilde{\vec{w}}_j$ = separate context vector for word j
- $b_i, \tilde{b}_j$ = bias terms
- $f(X_{ij})$ = weighting function

Weighting Function:

GloVe vs. Word2Vec

Word2Vec (Skip-gram):

- Local context windows
- Online training
- Predicts context from word
- Stochastic updates
- Negative sampling trick
- Each occurrence matters

Advantages:

- Works well on small corpora
- Can train incrementally
- Captures local patterns

GloVe:

- Global co-occurrence matrix
- Batch training
- Factorizes co-occurrence matrix
- Deterministic
- Weighted least squares
- Aggregates all occurrences

Advantages:

- Faster training (large corpora)
- Better statistical efficiency
- More interpretable

GloVe in Practice

```
import gensim.downloader as api
import numpy as np

# Load pre-trained GloVe embeddings
# Options: glove-wiki-gigaword-50, -100, -200, -300
# Or: glove-twitter-25, -50, -100, -200
glove = api.load("glove-wiki-gigaword-100")

# Use just like Word2Vec
similar = glove.most_similar('computer', topn=5)
print(similar)

# Analogies
result = glove.most_similar(
 positive=['france', 'berlin'],
 negative=['paris'],
```


continued...

GloVe in Practice

```
 topn=1
)
print(result) # Should be close to 'germany'

# Vector arithmetic
king = glove['king']
man = glove['man']
woman = glove['woman']
result_vec = king - man + woman

# Find closest word
closest = glove.similar_by_vector(result_vec, topn=1)
print(closest) # Should be close to 'queen'

# Compute similarity
similarity = glove.similarity('cat', 'dog')
```


...continued...

GloVe in Practice

```
print(f"Similarity: {similarity:.3f}")
```

Winter 2026

...continued

FastText: Enriching with Subword Information

The Problem with Word2Vec and GloVe:

Out-of-Vocabulary (OOV) Problem

- Word2Vec/GloVe: One vector per word
- New word?
- Misspelling?
- Rare morphological form?

Example:

- Have: "running", "runner"
- Need: "runnable" $\rightarrow$

FastText: Character N-grams

Key Idea: Break words into character n-grams

Example: "where" (with n=3)

Character trigrams: <wh, whe, her, ere, re>

Plus: <where> (the whole word)

Word vector:

$$\vec{w}_{\text{where}} = \frac{1}{6}\left(\vec{z}_{<\text{wh}} + \vec{z}_{\text{whe}} + \vec{z}_{\text{her}} + \vec{z}_{\text{ere}} + \vec{z}_{\text{re}>} + \vec{z}_{<\text{where}>}\right)$$

Advantages:

- Handles OOV words!

Example OOV:

Have trained on: "run", "running"

FastText: Morphology in Action

How FastText handles related words:

```
# Word: "unhappiness" broken into character 3-grams:
```

This is why FastText excels at morphologically rich languages!

Language	Morphology	FastText Advantage
English	Low	Moderate

FastText in Practice

```python
from gensim.models import FastText
import gensim.downloader as api

# Load pre-trained FastText model
# Available in 157 languages!
fasttext_model = api.load('fasttext-wiki-news-subwords-300')

# Works for known words
vec_cat = fasttext_model['cat']

# Also works for unknown words! (OOV)
vec_unknownword = fasttext_model['unknownword'] # Still get a vector!

# Even works for misspellings (somewhat)
vec_misspelling = fasttext_model['computr'] # Close to "computer"
```


continued...

FastText in Practice

```
16
17  # Train your own FastText model
18  sentences = [['the', 'cat', 'sat'], ['the', 'dog', 'ran']]
19
20  ft_model = FastText(
21   sentences,
22   vector_size=100,
23   window=5,
24   min_count=1,
25   min_n=3, # minimum n-gram length
26   max_n=6, # maximum n-gram length
27   word_ngrams=1 # use word + ngrams
28  )
29
30  # Check similar words
```


...continued...

FastText in Practice

```
similar = ft_model.wv.most_similar('cat', topn=5)

# Morphological awareness
# 'running', 'runner', 'runnable' will be close
```

Winter 2026

...continued

Embedding Methods Comparison

LDA	**2003**	**Probabilistic**		**Slow**	**Topic modeling**
Word2Vec	2013	Neural (local)		Fast	General NLP
GloVe	2014	Hybrid (global)		Fast	Large corpora

Evaluating Word Embeddings

Intrinsic Evaluation:

1. Word Similarity

- WordSim-353 dataset
- SimLex-999 dataset
- Spearman correlation with human judgments

2. Word Analogies

- Google Analogy Dataset (19k questions)
- Semantic: *Athens:Greece::Baghdad:Iraq*
- Syntactic: *good:better::bad:worse*

Extrinsic Evaluation:

Use embeddings as features in downstream tasks:

- **Sentiment Analysis**
- **Named Entity Recognition**
- **POS Tagging**
- **Machine Translation**
- **Question Answering**
- **Text Classification**

Key Insight

Limitations of Static Embeddings

The fundamental problem: One vector per word type

Example: Polysemy

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane will **bank** left" (tilt)

All get the ! This conflates different meanings.

Other Limitations:

- No compositional semantics
- "hot dog" ≠ "hot" + "dog"
- Bias in embeddings

The Solution:

- Different vector per occurrence
- Consider sentence context
- ELMo, BERT, GPT, ...

Bias in Word Embeddings

Embeddings learn the biases in their training data

Gender Bias Examples

- $\vec{\text{man}} : \text{computer } \vec{\text{programmer}} :: \vec{\text{woman}} : \vec{\text{homemaker}}$
- $\vec{\text{man}} : \vec{\text{doctor}} :: \vec{\text{woman}} : \vec{\text{nurse}}$
- "man" more associated with "career", "woman" with "family"

Other Biases:

- Racial/ethnic stereotypes

Bias Detection: Code Example

```
import gensim.downloader as api
model = api.load('word2vec-google-news-300')

# Measure gender bias: which words are closer to "man" vs "woman"?
def gender_bias_score(word):
 """Positive = closer to man, Negative = closer to woman"""
 return model.similarity(word, 'man') - model.similarity(word, 'woman')

occupations = ['doctor', 'nurse', 'engineer', 'teacher',
 'programmer', 'secretary', 'scientist', 'receptionist']

for job in occupations:
 score = gender_bias_score(job)
 direction = "→ man" if score > 0 else "→ woman"
 print(f"{job:12}: {score:+.3f} {direction}")

# Output:
```


continued...

Bias Detection: Code Example

```
# doctor : +0.089 → man
# nurse : -0.109 → woman
# engineer : +0.078 → man
# teacher : -0.046 → woman
# programmer : +0.091 → man
# secretary : -0.107 → woman
```

These biases reflect stereotypes in the training data (news articles)!

...continued

Debiasing Approaches

How can we reduce bias in embeddings?

1. Post-processing:

- Identify bias subspace
- Neutralize: Remove bias from neutral words
- Equalize: Ensure equal distances

2. Training-time:

- Counterfactual data augmentation
- Adversarial debiasing
- Constrained optimization

Challenges:

- Hard to define "bias"
- May harm performance
- Bias is multidimensional
- Not a complete solution
- Ethical considerations

Important

Debiasing is an active research area. No perfect solution yet!

Best practice:

- Medical: PubMed, clinical notes
- Legal: case law, contracts
- Social media: tweets, posts
- Can significantly improve performance

3. **Choose dimensions wisely**

- More dims = more expressiveness, more data needed
- 50-100d: small datasets, fast computation
- 200-300d: large datasets, better quality

4. **Use cosine similarity**

Loading Pre-trained Embeddings: Quick Reference

```
import gensim.downloader as api

# Word2Vec (Google News, 3B words, 300d)
w2v = api.load('word2vec-google-news-300')

# GloVe options
```

Pro tip: Start with glove-wiki-gigaword-100 for a good balance of speed and quality!

Real-World Applications

Search & Information Retrieval:

- Semantic search
- Query expansion
- Document ranking
- Question answering

Text Classification:

- Sentiment analysis
- Spam detection
- Topic classification
- Intent detection

Recommendation Systems:

- Product recommendations
- Content recommendations
- User similarity

Machine Translation:

- Word alignment
- Transfer learning
- Multilingual models

Creative Applications:

Discussion Question

Why do word analogies work so well?

The famous example:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

Think about:

- What does vector subtraction represent linguistically?
- Why does this capture semantic relationships?
- What are the limitations of this approach?
- Does this mean embeddings "understand" gender?

2. **GloVe (2014):** Combining count + prediction

- Global co-occurrence statistics
- Factorization objective
- Ratio of probabilities

3. **FastText (2017):** Subword information

Subword Information

Bias & Ethics:

- Bolukbasi et al. (2016). "Man is to Computer Programmer as Woman is to Homemaker?"
- Caliskan et al. (2017). "Semantics derived automatically from language corpora contain human-like biases"

Theory:

- Boleda, G. (2020). "Distributional Semantics and

Questions?

Next Lecture:

Contextual Embeddings: ELMo, Universal Sentence Encoder, BERT

One word, multiple meanings!

Winter 2026